

Trabajo Práctico Especial.

Alumnos: Villa Eliseo; Román Nahuel.

Universidad Nacional del Centro de la Provincia de Buenos Aires

Asignatura: Teoría de la Información.

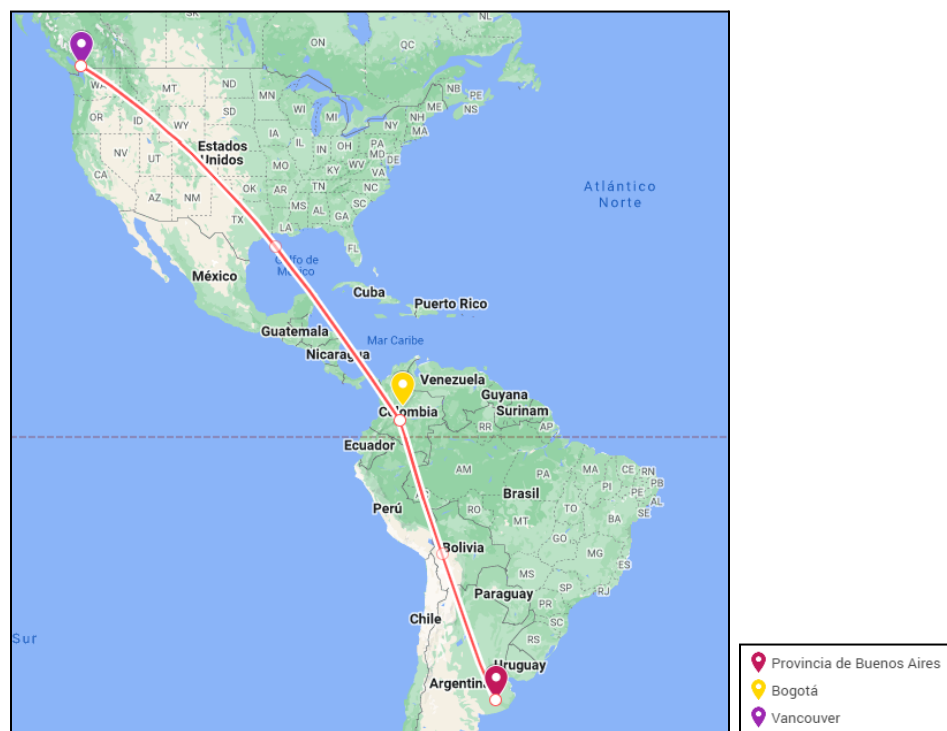
Profesor: Dra. Mariana del Fresno; Dr. Alejandro Clausse.

Fecha: 24/06/2024.

Resumen.

En este informe se expone el análisis estadístico y la implementación computacional de tres señales correspondientes a los valores diarios de temperatura promedio de tres ciudades diferentes (Buenos Aires, Bogotá y Vancouver) durante cierto periodo de tiempo. Se buscará la comprensión e interpretación de los resultados arrojados computacionalmente, como también la relación (si es que la hay) entre los valores de temperatura de las diferentes señales.

Los valores de temperatura están expresados en grados celsius, son números enteros y se los puede considerar como representativos del comportamiento de las temperaturas diarias en cada ciudad. La implementación de los algoritmos fué realizada con el lenguaje *Python* en el entorno de [Google Colab](#).



Análisis Estadístico.

En esta sección se plantea el análisis estadístico de cada señal, brindando un contexto a los resultados de *la media*, *el desvío estándar*, y *el factor de correlación cruzada* para cada señal.

Media y Desvío Estándar.

Los resultados obtenidos nos indican que la temperatura promedio en base a las señales de cada ciudad son:

Valor Medio de Temperatura		
<i>Buenos Aires.</i>	<i>Bogota.</i>	<i>Vancouver</i>
16.523	12.895	10.068

Interpretacion:

El desvío nos indica la variación o dispersión en la que los puntos de datos individuales difieren de la media. En el caso aquí planteado:

Valor Desvío Estándar		
<i>Buenos Aires</i>	<i>Bogotá</i>	<i>Vancouver</i>
5.982	1.063	5.624

Por ejemplo en Bogotá la temperatura se dispersa ± 1 °C de la media, es decir, los días en los que se midió la temperatura tuvieron valores muy similares. En cambio en Vancouver y Buenos Aires, el clima varió $\approx \pm 6$ °C de la media, osea que los valores de temperatura no fueron tan constantes como en Bogotá.

Factor de Correlación Cruzada.

El factor de correlación puede tomar un rango de valores de +1 a -1. Un valor cercano a 0 indica que no hay asociación entre las dos variables. Un valor mayor que 0 indica una asociación lineal positiva. Básicamente, a medida que aumenta el valor de una variable, también lo hace el valor de la otra. Un valor menor que 0 indica una asociación lineal negativa, lo que indica que cuando aumenta el valor de una variable, el valor de la otra tiende a disminuir.

$r_{s1,s2}$		
<i>Buenos Aires - Bogotá</i>	<i>Buenos Aires - Vancouver</i>	<i>Bogotá - Vancouver</i>
– 0.053	– 0.730	0.039

Interpretacion:

Al calcular el factor de correlación notamos que el valor que obtenemos entre la ciudad de Buenos Aires y Vancouver es negativo. Esto quiere decir que a medida que la temperatura aumenta en Buenos Aires disminuye en Vancouver. Por otra parte, entre la ciudad de Bogotá y Vancouver tenemos un valor positivo, lo que indica que si aumenta la temperatura en una ciudad también aumentará en la otra.

Tanto el factor de correlación entre Buenos Aires y Bogotá, como el de Bogotá y Vancouver son cercanos a cero, lo que indica que hay poca o nula asociación entre las temperaturas.

Señales como Fuente de Memoria.

Por cada una de las señales se creó una nueva señal que indica si la temperatura registrada de cada día es *baja*(menor a 10°), *alta*(mayor/igual a 20°), o *moderada*(menor a 20° y mayor/igual a 10°). Estas señales son modeladas como una *fuentes markoviana* y obteniendo su *matriz de pasaje* se calcula *entropía*(con y sin memoria), *conjunto de códigos* y *longitud total en bits*.

Entropía.Interpretacion:

La entropía es el promedio mínimo de preguntas binarias que se realizan para identificar un símbolo de la fuente. Para nuestro problema se plantearon dos cálculos de entropía. Por un lado la entropía con la fuente tratada como sin memoria(los símbolos son estadísticamente independientes) y la entropía condicional. A su vez, para futuros análisis se calculó la entropía de la fuente extendida de orden dos.

<i>Entropía</i>	<i>Buenos Aires</i>	<i>Bogotá</i>	<i>Vancouver</i>
Sin memoria H_1	1.426	0.034	1.155
Condicional H_{cond}	0.847	0.0319	0.451
Con memoria fuente extendida H_2	2.273	0.3719	1.606

Codificación mediante Huffman.

$\langle L_{\text{Huffman}} \rangle$ Fuentes sin memoria (original)		
<i>Buenos Aires</i>	<i>Bogotá</i>	<i>Vancouver</i>
1.489	0.999	1.507

1° Teorema de Shannon para fuentes **sin** memoria:

- Buenos Aires cumple el Teorema de Shannon
- Bogotá cumple el Teorema de Shannon
- Vancouver cumple el Teorema de Shannon

$\langle L_{\text{Huffman } 2} \rangle$ Fuentes extendidas con memoria de orden 2		
<i>Buenos Aires</i>	<i>Bogotá</i>	<i>Vancouver</i>
2.310	1.010	1.773

1° Teorema de Shannon para fuentes **con** memoria:

- Buenos Aires cumple el Teorema de Shannon
- Bogotá cumple el Teorema de Shannon
- Vancouver cumple el Teorema de Shannon

Pseudocódigo utilizado.

```
def calcular_matriz_pasaje:
    # Contamos las transiciones entre estados consecutivos
    para cada estado consecutivo:
        transiciones[estado_actual][estado_siguiete] += 1

    por cada estado de la señal:
        total_transiciones = ocurrencias totales del estado
        para cada estado siguiente partiendo del estado actual
            matriz_pasaje[estado][siguiete] = transiciones[estado][siguiete]
                                                / total_transiciones

    return matriz_pasaje
```

```
def vector_estacionario(matriz_pasaje, error_minimo):
    vector_ant
    vector_actual

    inicializar_vectores(vector_ant, vector_actual)

    while(existe_error(error_minimo)):
        for x in matriz_pasaje:
            for y in matriz_pasaje[x]:
                vector_actual[x] = (vector_ant[y]*matriz_pasaje[y][x]) + vector_actual[x]

        for s in vector_actual:
            resta = abs(vector_actual[s] - vector_ant[s])
            vector_ant[s] = vector_actual[s]
            vector_actual[s] = 0

    return vector_ant
```

```
def huffman_codificacion(fuente_extendida):
    codificacion = {}

    while(fuente_extendida[0][0] < 0.99999):
        ordenar(fuente_extendida)
        # Calcular la nueva probabilidad sumando las dos más pequeñas
        prob = nueva_probabilidad(fuente_extendida)
        nueva_cod = nueva_lista(prob)

        # para cada código en el primer elemento de fuente_extendida después del primero hacer
        # añadir "0" + código a nueva_codificacion
        añadir_cero(nueva_cod, fuente_extendida)

        # Añadir '1' a los códigos del segundo elemento y añadir a nueva_codificacion
        # para cada código en el segundo elemento de fuente_extendida después del primero
        añadir_uno(nueva_cod, fuente_extendida)

        # Eliminar los dos primeros elementos de fuente_extendida
        eliminar_primeros_dos_elementos(fuente_extendida)

        # Añadir la nueva combinación a fuente_extendida
        agregar_nueva_lista(fuente_extendida, nueva_cod)

        fuente_extendida.append(nueva_cod)

    codificacion = nueva_codificacion(fuente_extendida)

    return codificacion
```

Interpretación:

Si observamos la longitud promedio de la codificación de las señales mediante el método de Huffman notaremos que el valor es muy cercano a la entropía calculada en el inciso anterior, tanto para la fuente sin memoria como para la extendida, lo que denota que la codificación es muy buena pero que no es óptima.

Longitud en bits y Tasa de compresión.

Fuentes sin memoria (original)			
	<i>Buenos Aires</i>	<i>Bogotá</i>	<i>Vancouver</i>
<i>Longitud bits</i>	10338	6940	10468
Tasa compresión	20.714	31.971	18.692

	Fuentes extendidas con memoria de orden 2		
	<i>Buenos Aires</i>	<i>Bogotá</i>	<i>Vancouver</i>
Longitud bits	7996	3507	6117
Tasa compresión	26.781	63.267	31.988

Interpretación:

Para las fuentes originales la tasa de compresión es alta pero no mayor a las fuentes extendidas porque al no hacer agrupación de símbolos la longitud total en bits es mayor, por ende, en comparación la compresión es menor.

Pseudocódigo Utilizado:

```
def tasa_compresion(clasificacion, codificacion, orden, tam_datos_originales):
    i, tam_datos_comprimido = 0
    cant_simbolos = cantidad de simbolos de la señal

    while i < cant_simbolos
        si orden es 1
            simbolo = clasificacion[i]
        sino
            simbolo = clasificacion[i] concatenado con clasificacion[i + 1]

        tam_datos_comprimido = sumatoria del tamaño de codificacion[simbolo]
        i = i + orden

    tasa de compresion = tam_datos_original
```

Características del Canal.

En esta sección se analizan las características de un canal de información que transmite las temperaturas promedio registradas en una ciudad. Para ello se tiene una “señal enviada” con temperaturas de la ciudad de Buenos Aires y una “señal recibida”.

Pseudocódigo Utilizado:

```
def calcular_matriz_canal(entrada, salida):
    por cada par (estado_entrada, estado_salida) en agrupar(entrada, salida)
        cambio[estado_entrada][estado_salida] += 1

    por cada estado_entrada en cambio
        total_ocurrencias = ocurrencias totales del estado_entrada
        por cada estado_salida partiendo de estado_entrada
            matriz_canal[estado_entrada][estado_salida] = cambio[estado_entrada][estado_salida]
                                                                / total_ocurrencias

    return matriz_canal
```

Ruido e Información mutua del canal.

El ruido agrega información adicional no propia de la fuente sino del canal. Si con la entrada no puedo definir la salida entonces nos encontramos con ruido.

El valor de ruido calculado es 0.671, lo que significa que el canal distorsiona de manera significativa la entrada.

Por otra parte, la información mutua permite *cuantificar* el *ahorro entrópico* (ahorro de preguntas binarias) que se consigue cuando dos variables están acopladas. Con la información mutua podemos estudiar la correlación más allá que no sea un valor cuantitativo.

El valor de información mutua calculado es 0.779, lo que significa que tenemos un ahorro del promedio mínimo de preguntas binarias debido a la fuerte relación entre las fuentes ya que es la misma fuente (señales de Buenos Aires) con la diferencia que una atravesó un canal de información y la otra no.

Pseudocódigo Utilizado:

```
def calcular_ruido(distr_fuente_entrada, matriz_canal):
    ruido = 0

    para cada entrada en distr_fuente_entrada
        ruido_chiquito = 0
        probabilidad_entrada = distr_fuente_entrada[entrada]

        por cada salida en partiendo de entrada
            ruido_chiquito = ruido_chiquito - matriz_canal[entrada][salida] *
                | log2(matriz_canal[entrada][salida])

        ruido_chiquito = ruido_chiquito * probabilidad
        ruido = ruido + ruido_chiquito

    return ruido
```

```
def calcular_informacion_mutua(P, matriz_prob_condicional, ruido):
    Py[] = 0
    por cada X en matriz_prob_condicional # Columna
        por cada Y en matriz_prob_condicional[X] # Fila
            Py[Y] = Py[Y] + matriz_prob_condicional[X][Y] * P(X) # Sumo las ramas

    H(Y) = calcular_entropia(Py[])

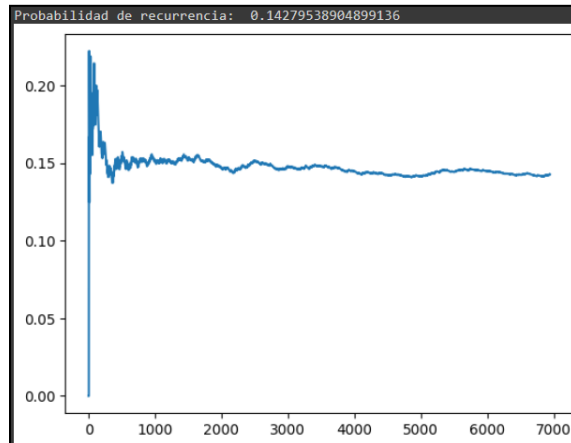
    retornar (H(Y) - ruido)
```

Simulación computacional.

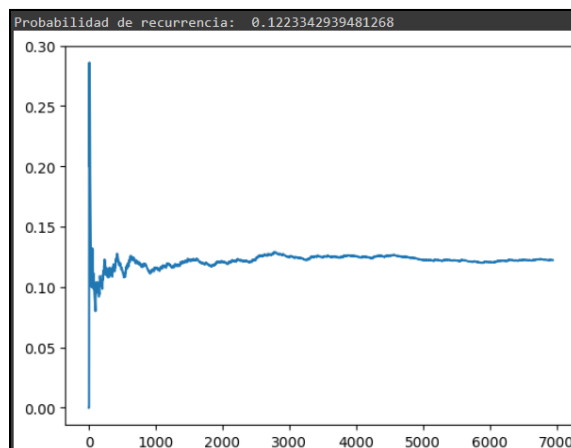
Mediante el método de las simulaciones de Montecarlo se buscó obtener la probabilidad de dos apariciones consecutivas de un mismo símbolo con hasta una cantidad estipulada de símbolos distintos entre las apariciones. Para esto se generó aleatoriamente la entrada y la salida correspondiente del canal, y se calculó la probabilidad a la salida del canal.

Esta simulación se hizo para diferentes símbolos y diferentes cantidades (N) de símbolos distintos entre las apariciones utilizando las temperaturas de Buenos Aires, y se graficaron los gráficos de convergencia.

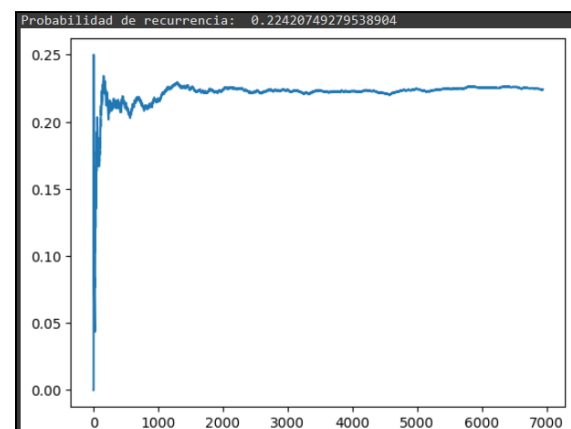
- **Símbolo:** “A”
- **N:** 3



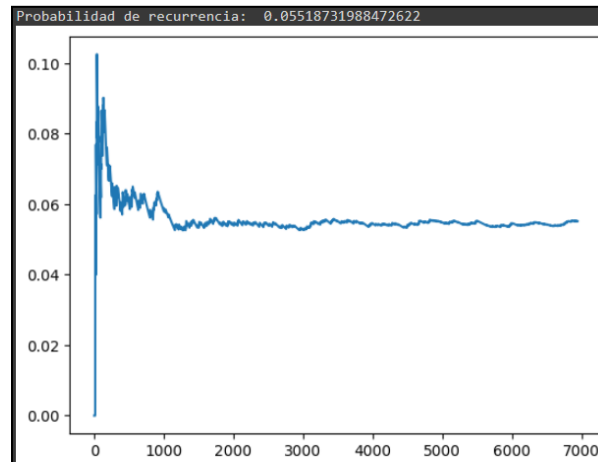
- **Símbolo:** “A”
- **N:** 2



- **Símbolo:** “M”
- **N:** 3



- **Símbolo:** “B”
- **N:** 2



Interpretación.

La convergencia en los gráficos indica la estabilidad de la probabilidad estimada a medida que aumenta el número de iteraciones, como podemos observar en los gráficos, siempre se alcanza un valor de probabilidad estable.

Pseudocódigo Utilizado.

```
def media_recurrencia(simbolo buscado, epsilon, MIN_ITERACIONES, distribucion,
                     matriz_canal, n_simbolos_dist):

    (inicializacion de variables)

    mientras ((prob_ant <> 0 y la probabilidad no converge) o (t_actual < MIN_ITERACIONES))
        t_actual = t_actual + 1
        simbolo_salida = generar_salida(matriz_canal, simbolo_entrada)

        si (0 < cont <= n_simbolos_dist y simbolo_salida es igual a simbolo_buscado)
            exitos = exitos + 1

        si (simbolo_salida == simbolo):
            cont = 0
            flag_simbolo = True
        sino si(flag_simbolo es True)
            cont = cont + 1

        prob_ant = prob_actual
        prob_actual = exitos / t_actual

        simbolo_entrada = generar_entrada(distribucion)
        agregar prob_actual a probabilidades[]
        agregar t_actual a iteraciones[]
    return (probabilidades, iteraciones)
```

Conclusiones.

El trabajo presenta un análisis de las características climáticas representadas por las señales de temperatura en las ciudades de Buenos Aires, Bogotá y Vancouver.

Los resultados obtenidos además de ser relevantes para la comprensión del comportamiento climático, son también importantes para la aplicación práctica de técnicas de codificación y relación de fuentes de información en un contexto real.